

Linea - Yield Manager

1 Executive Summary

2 Scope

2.1 Objectives

3 System Overview

3.1 Contracts

4 Security Specification

4.1 Actors

4.2 Trust Model

5 Findings

5.1 If No Yield Provider Has Enough Liquidity for an LST Withdrawal, the User's Message Is Stuck Major ✓ Fixed

5.2 `unstakePermissionless` DOS Attack Using Valid but Stale Container Proofs That Blocks Future `unstakePermissionless` Calls. Major ✓ Fixed

5.3 `unstakePermissionless` Forced Unstake Attack Using Valid but Stale Container Proofs That Unstake Any Validator Top-Ups or Increases in Balances Major ✓ Fixed

5.4 If `unstake` Is Pending While `unstakePermissionless` Can Be Called the System May Unstake More Than the System Intends Medium Acknowledged

5.5 Whale-Driven Reserve Depletion Forcing Excessive LST Liability Medium ✓ Fixed

5.6 `claimMessageWithProofAndWithdrawLST` Requires `msg.sender==params._to` Which Is Inconsistent With `claimMessageWithProof` Behavior Medium ✓ Fixed

5.7 A Timed Large-Holder Attack Can Cause the System to Over-Unstake Whenever `unstake` Is Called Medium ✓ Fixed

5.8 Over-Unstaking Due to Greedy Decrementing of `pendingPermissionlessUnstake` Medium ✓ Fixed

5.9 Inconsistent `_pauseStakingIfNotAlready` Conditions in Withdrawal Functions Medium ✓ Fixed

5.10 Lido External Shares Limit May Block Emergency LST Withdrawals Medium ✓ Fixed

5.11 `_hashMessageWithEmptyCalldata` Does an Unnecessary 0x Byte Insert Minor ✓ Fixed

5.12 Unnecessarily Tight Boundary on a Check When Withdrawing With LST Tokens Minor ✓ Fixed

5.13 Unnecessary 0-Address Checks Minor ✓ Fixed

5.14 Inconsistent 0-Address Checking for `_yieldManager` Minor ✓ Fixed

1 Executive Summary

This report presents the results of our engagement with **Linea** to review **Linea Yield Manager**.

The review was conducted from **November 10, 2025** to **December 12, 2025** by **George Kobakhidze** and **Arturo Roura**.

The Linea Yield Manager protocol is designed to stake ETH bridged from Ethereum L1 to the Linea rollup into Ethereum beacon chain validators via Lido’s v3 stVault infrastructure, and to distribute the resulting staking yield back to users on Linea (L2). The audit scope covered the core yield orchestration, the rollup integration used to route funds and report yield across domains, and the message service components used to support cross-chain accounting and claims.

A central security and product constraint of this design is liquidity: once bridged ETH is deployed into staking, users still expect timely withdrawals on L1/L2. The protocol addresses this by maintaining configurable withdrawal reserves and by introducing permissionless flows that allow users (or third parties) to help restore solvency/liveness in edge cases (e.g., triggering unstaking when reserve levels are insufficient). While these mechanisms reduce liquidity risk, they also expand the state space and introduce edge cases driven by diverse user behaviors, asynchronous withdrawals, and cross-protocol dependencies (Lido and the beacon chain).

Overall, the reviewed codebase demonstrates a consistent structure and clear documentation, and no widespread low-level implementation flaws were identified within the scope of the audit. Many issues are mitigable operationally because the system includes privileged controls to pause, configure, and replenish liquidity, but this also means the protocol relies on strong operational security: multiple powerful roles can materially affect user funds and protocol behavior, and the upgradeability/administrative surface must be treated as a primary trust assumption.

Update January 20th 2026: Upon the completion of the audit, we have verified that the commit `25e323d055dec40ef167a190c71c30aa9bf92c23` contains the necessary fixes for issues found during the audit. This commit also contains the deployment artifacts in `contracts/deployments/bytecode/2026-01-14/*` the bytecode within which matches the bytecode in the deployment artifacts we built locally. Finally, we also confirm that the same bytecode artifacts were merged into the `main` branch in the commit `9c16ce37d2dca6e6106cdc83202979c138ee0130`.

Update February 12th 2026: Following the deployment of the yield management system contracts reviewed in this engagement and prior to any upgrades, we have verified that the bytecode deployed on Ethereum mainnet exactly matches the expected runtime bytecode specified in the local build artifacts from the ‘main’ branch at the following addresses:

- YieldManager Implementation: `0x751236A1aFC11B7F1A7630fe87b0Bd96AC5203C4`
- ValidatorContainerProofVerifier: `0x2309C45d44105928b483f608dD6140Fb65f3EBde`
- LidoStVaultYieldProviderFactory: `0xE4FC9F1A8cB97fEAd3C2b37c11AD5B1C2eF73959`
- LidoStVaultYieldProvider: `0x486D8cADc10489B30b64c890aEc747F1220eEEC3`
- LineaRollup Implementation: `0x04728BF704a716C26F9EF4085013b760AC885631`

This verification confirms that the deployed contracts correspond exactly to the audited source code.

Update August 3rd 2026: The Linea team has identified an issue in the generation of the yield message hash. The hash is computed with the `LineaRollup` as the sender, but the event uses the Yield Manager. Message claiming is still possible, but it breaks existing systems because of the mismatch. A fix has was created in [PR 3654](#) and merged in commit `82af194f8c680abd238bdd5f7c85acabf073bdbb`. We have verified that the posted byte code in `contracts/local-deployments-artifacts/dynamic-artifacts/LineaRollupV8.json` matches what we built locally as well.

2 Scope

This review focused on the following repository and code revision:

- [Consensys/linea-monorepo](#) @ `4360e31fa8c8259f6b6cbccdd45bb94032a792c6`

The detailed list of files in scope can be found in the [Appendix](#).

2.1 Objectives

Together with the **Linea team**, we identified the following priorities for this review:

- Correctness of the implementation, consistent with the intended functionality and without unintended edge cases.
- Identify known vulnerabilities particular to smart contract systems, as outlined in our [Smart Contract Best Practices](#), and the [Smart Contract Weakness Classification Registry](#).
- LST withdrawal is available for users during liquidity shortfalls and accounting is done appropriately.
- Staking operations progress along the intended routes including pausing, withdrawal, and ossification.
- Permissionless operations such as unstaking are available for users and do not inhibit the protocol’s functions.

of Scope]

✓ Fixed

5.16 Some @dev Comments

Explicitly Call Out the Required Role to Call the Function, Some

Don't

✓ Fixed

6 Self Reported by the Linea Team

6.1 Stale withdrawableValue

Calculation Blocks All Withdrawals When LST Liabilities Exist

Major

✓ Fixed

6.2 Missing Zero-Balance Check in progressPendingOssification Blocks Ossification Completion

Minor

✓ Fixed

Appendix 1 - Files in Scope

Appendix 2 - Disclosure

A.2.1 Purpose of Reports

A.2.2 Links to Other Web Sites from This Web Site

A.2.3 Timeliness of Content

The Linea Yield Manager provides a native yield infrastructure integrating the Linea L2 bridge with Lido’s Staking Vault V3 protocol. It enables L1 ETH held in Linea’s withdrawal reserve to earn staking yield while upholding strict liquidity guarantees for bridge withdrawals. The system uses a delegatecall-based architecture composed of four deployable contracts.

Yield operations follow a structured, multi-phase workflow:

- **Reserve Management:** `YieldManager` monitors minimum and target withdrawal-reserve thresholds and determines when excess ETH can be safely deployed for staking.
- **Yield Generation:** Surplus ETH is routed to `LidoStVaultYieldProvider` (via delegatecall), which stakes funds through Lido’s VaultHub/Dashboard/StakingVault architecture to earn beacon-chain rewards.
- **Yield Reporting:** Accrued yield is derived from Lido’s `totalValue()` minus system obligations and communicated to Linea L2 via cross-chain messaging.
- **Liquidity Provisioning:** Operator-initiated or permissionless unstaking pathways return staked ETH to the reserve during heightened withdrawal demand, with permissionless flows leveraging EIP-4788 beacon-chain proof verification.
- **Graceful Exit:** A staged ossification procedure enables full withdrawal from Lido, including voluntary detachment from VaultHub and recovery of the 1 ETH connect deposit.

3.1 Contracts

YieldManager

The `YieldManager` contract orchestrates native yield generation by managing withdrawal reserves and coordinating staking operations across multiple yield providers. It reports accrued yield to Layer 2 recipients using Linea’s cross-chain messaging infrastructure.

Key Features:

- **Upgradeable Design:** UUPS proxy pattern with OpenZeppelin `AccessControlUpgradeable` .
- **Namespaced Storage:** ERC-7201 implementation prevents storage collisions across upgrades.
- **Role-Based Access Control:** Distinct roles for managing providers, reserves, reporting, staking, ossification, and L2 recipient management.

Reserve Management:

- **Dual Threshold System:**
 - *Minimum reserve:* Safety buffer, dynamically calculated.
 - *Target reserve:* Ensures sufficient liquidity for bridge withdrawals.
- **Continuous Monitoring:** Tracks reserve levels and triggers replenishment when necessary.
- **Fund Flow Management:** Handles ETH transfers between `L1MessageService` and yield providers.

Yield Provider Coordination:

- Executes provider logic via delegatecall in `YieldManager` ’s storage context.
- Supports multi-provider setup with per-provider accounting (`userFunds` , `yieldReportedCumulative`).
- Lifecycle management: addition, active operations, ossification, and removal.
- Access restrictions prevent direct calls to provider implementations.

Yield Reporting:

- Reports yield to L2 via `ILineaRollupYieldExtension.reportNativeYield()` .
- Maintains cumulative yield accounting per provider.
- Restricts destinations to approved L2 contracts.
- Emits events for transparency (`NativeYieldReported`).

Configuration & Emergency Management:

- Reserve thresholds, provider registry, L2 allowlist, and emergency removal are configurable.
- Emergency controls enable immediate provider removal without standard checks.

LidoStVaultYieldProvider

Integrates with Lido Staking Vault V3 and handles staking, validator withdrawals, stETH liability management, and ossification. Runs in `YieldManager` ’s storage via delegatecall.

Key Features:

- **Delegatecall Execution:** Shares `YieldManager` storage context using ERC-7201.
- **Immutable Addresses:** Lido protocol contracts set at deployment.
- **Stateful Operations:** Tracks ossification, LST liabilities, and staking pauses.

Operations:

- **Staking:** Funds vault, pays stETH liabilities, updates internal accounting, emits `YieldProviderFunded` .
- **Yield Reporting:** Calculates yield considering user funds, liabilities, and fees, then prepares for cross-chain reporting.
- **Unstaking:**
 - *Operator-driven:* Triggers validator withdrawals via Lido interfaces.
 - *Permissionless:* Validates proofs, restricts partial withdrawals, enforces minimum validator activation and balance.
- **Withdrawals:** Retrieves ETH for `YieldManager` , ensuring LST liabilities are settled.

- Initiation: Settles LST liabilities and disconnects gracefully.
- Progression: Transfers vault ownership, ossifies, and finalizes staged balances.
- Removal: Only possible after ossification; storage cleared and event emitted.

Pause Controls: Granular pause system for staking, unstaking, permissionless actions, donations, and reporting. Automatically enforced during ossification.

ValidatorContainerProofVerifier

Verifies Ethereum beacon chain validator proofs to support secure permissionless unstaking.

Features:

- Uses SSZ Merkle proof verification and EIP-4788 beacon block roots.
- Ensures validators are active, meet minimum balance requirements, and prevents unstaking of slashed or underfunded validators.
- Cryptographically robust and tamper-resistant.

LidoStVaultYieldProviderFactory

- Deploys new `LidoStVaultYieldProvider` instances with preconfigured Lido addresses.
- Standard `new` deployment (no CREATE2), emitting creation events for transparency.

4 Security Specification

This section describes, from a security perspective, the expected behavior of the Linea Yield Manager system under review. It is not a substitute for documentation. The purpose of this section is to identify specific security properties that were validated by the scoping team.

4.1 Actors

The relevant actors are listed below with their respective abilities:

System Administrators

Proxy Admin Owner. The `YieldManager`, `L1MessageService`, and `LineaRollupYieldExtension` contracts are upgradeable. The Proxy Admin Owner is the address that can perform such upgrades. his role has complete control over contract logic and storage.

Default Admin. There is a Default Admin assigned to the `YieldManager` contract. This admin manages all roles in the system, including the Default Admin role itself. They can grant and revoke any role to any address.

Operational Roles

YIELD_PROVIDER_STAKING_ROLE. This role can transfer ETH from the `L1MessageService` withdrawal reserve to the `YieldManager` via `LineaRollupYieldExtension.transferFundsForNativeYield()`. They can also fund specific yield providers by calling `YieldManager.fundYieldProvider()`, which moves ETH from `YieldManager` to staking providers. This role is critical as it controls the flow of funds from the reserve into yield-generating positions.

YIELD_PROVIDER_UNSTAKER_ROLE. This role can initiate unstaking operations from yield providers by calling `YieldManager.unstake()`. They can also withdraw ETH directly from yield providers via `withdrawFromYieldProvider()`, rebalance system liquidity through `addToWithdrawalReserve()` and `safeAddToWithdrawalReserve()`, and transfer funds to the reserve via `transferFundsToReserve()`. This role is essential for liquidity management, withdrawal reserve replenishment, and maintaining adequate system reserves for user withdrawals.

YIELD_REPORTER_ROLE. This role can request yield reports by calling `YieldManager.reportYield()`, which calculates net yield from a yield provider and emits a synthetic cross-chain message to distribute yield to L2 recipients. The yield report is a critical financial operation that determines how much yield is credited to L2 users.

STAKING_PAUSE_CONTROLLER_ROLE. This role can pause and unpause beacon chain deposit operations for specific yield providers via `YieldManager.pauseStaking()` and `unpauseStaking()`. This is an emergency control to stop new deposits while allowing existing operations to continue.

OSSIFICATION_INITIATOR_ROLE. This role can initiate the ossification process for a yield provider by calling `YieldManager.initiateOssification()`. Ossification is the graceful exit process from a yield provider, eventually leading to full withdrawal of funds and removal of the provider from the system.

OSSIFICATION_PROCESSOR_ROLE. This role can progress through the multi-step ossification process by calling `YieldManager.progressPendingOssification()`. This includes disconnecting from Lido vaults, accepting ownership transfers, ossifying staking vaults, and unstaging funds.

WITHDRAWAL_RESERVE_SETTER_ROLE. This role can update the withdrawal reserve parameters (minimum and target thresholds, both percentage-based and absolute amounts) via `YieldManager.setWithdrawalReserveParameters()`. These parameters are critical for ensuring the `L1MessageService` maintains sufficient liquidity for user withdrawals.

SET_YIELD_PROVIDER_ROLE. This role can add new yield providers via `YieldManager.addYieldProvider()`, remove yield providers via `removeYieldProvider()` (requires zero user funds), or perform emergency removal via `emergencyRemoveYieldProvider()`. Managing yield providers affects where system funds are allocated.

SET_L2_YIELD_RECIPIENT_ROLE. This role can add and remove addresses that are authorized to receive yield reports on L2 via `YieldManager.addL2YieldRecipient()` and `removeL2YieldRecipient()`. Only registered L2 recipients can receive yield distributions.

SET_YIELD_MANAGER_ROLE. This role exists on the `LineaRollupYieldExtension` contract and can update the `YieldManager` address via `setYieldManager()`. This is a critical configuration parameter that determines which contract can receive funds from the

Pause Type Role Holders. Various roles control type-specific pausing (e.g., `PAUSE_NATIVE_YIELD_STAKING` , `PAUSE_L1_L2`). These roles can activate emergency stops for specific subsystems without pausing the entire protocol. Unpause roles are separately assigned and may differ from pause role holders.

Users

Regular Users. Users interact with the system primarily through the `L1MessageService` for deposits and withdrawals. They can claim messages from L2 (withdrawal requests) via `L1MessageService.claimMessageWithProof()` , which may trigger ETH transfers from the withdrawal reserve. Users do not have direct interaction with `YieldManager` or yield providers.

Emergency Claimants. When the `L1MessageService` withdrawal reserve has insufficient ETH to fulfill a claim, users can invoke `LineaRollupYieldExtension.claimMessageWithProofAndWithdrawLST()` to receive stETH (Lido Staked ETH) instead. This function requires the caller to be the message recipient and temporarily enables LST withdrawal through a transient flag. This provides a safety valve during liquidity shortfalls.

Permissionless Unstakers. Any user can submit a permissionless unstake request via `YieldManager.unstakePermissionless()` if they provide a valid beacon chain validator container proof. This proof demonstrates that a validator associated with the yield provider has an active status and appropriate withdrawal credentials. Users must pay the unstake fee (sent with the transaction) to cover beacon chain withdrawal costs.

4.2 Trust Model

In any system, it’s important to identify what trust is expected/required between various actors. For this review, we established the following trust model:

Administrative Trust

Proxy Admin Owner. The ability to upgrade contracts gives the Proxy Admin Owner complete control over code and storage. If the Proxy Admin Owner is a timelock contract, upgrade execution is delayed, providing transparency and an opportunity for stakeholders to react. Users trust that upgrades are thoroughly reviewed and do not introduce malicious logic or storage corruption.

Default Admin. As the Default Admin can manage all roles (including granting and revoking Default Admin itself), this address has ultimate control over the system’s operational permissions. Users trust this role will be held by a decentralized security council that acts in the protocol’s best interest and does not abuse role assignment to enable exploits or censorship.

Various Privileged Configuration and Function Roles. As described above, there is a multitude of permission roles that may change configurations associated with staking, as well as progress staking operations such as funding and withdrawing. Separating them allows for a trust-minimized principle of least privilege approach, but it is still required that none of those roles are compromised or malicious for the protocol to operate correctly.

Protocol Trust

Linea L1MessageService The system fundamentally depends on the `L1MessageService` and its associated offchain components for cross-chain communication, ETH custody, and message proof verification. The message service acts as the withdrawal reserve custodian and validates Merkle proofs for user withdrawal claims via `claimMessageWithProof()` . Users trust that the message service correctly implements proof verification against L2 state roots, maintains accurate message numbering and rolling hash calculations, and will not censor legitimate withdrawal requests. The yield system relies on the message service to emit synthetic `MessageSent` events for yield distribution (via `reportNativeYield()`) and to execute the emergency LST withdrawal path (`claimMessageWithProofAndWithdrawLST()`) when reserves are insufficient. Any compromise of the message service’s proof validation, state root management, or cross-chain messaging could allow unauthorized fund withdrawals, prevent legitimate yield distributions, or block user access to their deposits. The system assumes the message service operators maintain the integrity of the L1↔L2 bridge and that L2 state commitments accurately reflect the true L2 state.

Lido Protocol. The system relies on Lido contracts to stake ETH, calculate vault values and liabilities, process validator withdrawals, and settle fees. Any bugs, exploits, or unexpected behavior changes in Lido could propagate to the Linea system. Lido governance could modify fee structures, pause vaults, or upgrade contracts in ways that impact yield calculations. Users trust that Lido is well-maintained, audited, and will not make breaking changes without notice. The system includes some defensive measures (try-catch blocks, external liability tracking) but cannot fully insulate itself from Lido risks.

Additionally, there are two mechanisms that could suppose a risk:

- Mentioned in [an issue](#), there are limitations to stETH minting established by Lido that are invisible to this system, but crucial for providing an effective liquidity layer between staked funds and the L1 message service reserve.
- The `withdrawableValue` used in this system’s accounting has intricacies, like the fact that stETH minting from Staking Vaults could reduce the Yield Provider’s `withdrawableValue` if a large amount of stETH has been minted relative to the total amount of funds managed by the staking vault.

Validator Container Proof Verifier. This contract verifies beacon chain proofs to authorize permissionless unstakes. Incorrect proof validation could allow malicious actors to trigger unauthorized validator exits or reject legitimate unstake requests. Users trust that the proof verification correctly implements beacon chain specifications (SSZ encoding, BLS signatures, Merkle proofs, generalized indices) and has been thoroughly tested against real beacon chain data.

Ethereum Beacon Chain. The system assumes beacon chain validators operate correctly and that validator withdrawals process as expected. Validator slashing, prolonged exit queues, or consensus issues could delay fund returns and affect yield. Users trust that beacon chain mechanics function as designed and that the protocol’s validator operators maintain good standing.

5 Findings

Each issue has an assigned severity:

- **Critical** issues are directly exploitable security vulnerabilities that need to be fixed.

exploited. All major issues should be addressed.

- Medium

 issues are objective in nature but are not security vulnerabilities. These should be addressed unless there is a clear reason not to.
- Minor

 issues are subjective in nature. They are typically suggestions around best practices or readability. Code maintainers should use their own judgment as to whether to address such issues.
- Issues without a severity are general recommendations or optional improvements. They are not related to security or correctness and may be addressed at the discretion of the maintainer.

5.1 If No Yield Provider Has Enough Liquidity for an LST Withdrawal, the User’s Message Is Stuck

Major

✓ Fixed

Resolution
Acknowledged. The Linea team has operational mitigations in place: (1) they plan to use only a single yield provider, and (2) they will run backend automation to regularly rebalance funds to ensure the L1 Message Service doesn’t fall into a sustained deficit.

Description

During liquidity shortfalls, the system allows users to mint LST tokens against the staked ETH to satisfy bridge requests via `claimMessageWithProofAndWithdrawLST()` . To do so, users claiming bridge messages via LST withdrawal must specify a particular yield provider, but if that provider lacks sufficient userFunds to mint the required LST amount, the transaction reverts with `LSTWithdrawalExceedsYieldProviderFunds` :

contracts/contracts/yield/YieldManager.sol:L839-L857

```
function withdrawLST(
    address _yieldProvider,
    uint256 _amount,
    address _recipient
)
    external
    whenTypeAndGeneralNotPaused(PauseType.NATIVE_YIELD_PERMISSIONLESS_ACTIONS)
    onlyKnownYieldProvider(_yieldProvider)
{
    if (msg.sender != L1_MESSAGE_SERVICE) {
        revert SenderNotL1MessageService();
    }
    if (!ILineaRollupYieldExtension(L1_MESSAGE_SERVICE).isWithdrawLSTAllowed()) {
        revert LSTWithdrawalNotAllowed();
    }
    YieldProviderStorage storage $$ = _getYieldProviderStorage(_yieldProvider);
    if (_amount > $$userFunds) {
        revert LSTWithdrawalExceedsYieldProviderFunds();
    }
}
```

However, since the message was already sent with a specified amount on the L2 Message Service, it is no longer possible for the user to change the amount:

contracts/contracts/messageService/l2/v1/L2MessageServiceV1.sol:L66

```
function sendMessage(address _to, uint256 _fee, bytes calldata _calldata) external payable {
```

contracts/contracts/messageService/l2/v1/L2MessageServiceV1.sol:L94

```
bytes32 messageHash = MessageHashing._hashMessage(msg.sender, _to, postmanFee, valueSent, messageNumber, _calldata);
```

As a result, if there is no single yield provider that can satisfy the full request, it will never be possible for this message to be claimed via the `claimMessageWithProofAndWithdrawLST()` route. The user will be forced to wait until sufficient liquidity is restored on the L1 Message Service.

Recommendation

Consider implementing a mechanism to choose multiple yield providers to satisfy LST withdrawals.

5.2 unstakePermissionless DOS Attack Using Valid but Stale Container Proofs That Blocks Future unstakePermissionless Calls.

Major

✓ Fixed

Resolution
Fixed in PR 1879 by introducing the following fixes: - requiring there to be a minimum of a <code>8192</code> slot difference between any two proven containers during permissionless unstaking requests, ensuring that any subsequent unstaking requests have container proofs that register previous actions - supplementing unstaking requests with pending withdrawal containers that allow to calculate the currently pending withdrawal amounts for a specific validator

target deficit and the current funds meant to rebalance it. This removes a DOS vector where a user could specify a really small amount to unstake, blocking a future unstaking request for the specified validator for 8192 slots.

Description

The `ValidatorContainerProofVerifier` validates beacon chain proofs against EIP-4788 beacon roots that does not enforce proof recency beyond the 8191-block (~27 hour) history window. This allows attackers to submit valid but stale proofs that reflect a validator’s historical **high** balance, while the validator’s current balance is **significantly lower** (due to slashing, partial withdrawals, or balance reductions). This creates unfulfillable withdrawal requests that permanently inflate `pendingPermissionlessUnstake` , causing a denial-of-service on legitimate permissionless unstake operations.

The attack exploits the timing window when validators experience balance reductions. An attacker can:

1. Identify a validator that experienced a balance reduction (e.g., from 2000 ETH to 100 ETH)
2. Generate a proof from before the reduction (up to 27 hours old, when balance was 2000 ETH)
3. Submit `unstakePermissionless` with `amounts[0]` set to the historical high balance (2000 ETH)
4. The proof verification passes (stale but valid), computing `maxUnstakeAmountGwei = Math256.min(2000e9, effectiveBalance - 32e9) ≈ 1968 ETH`
5. The system increments `pendingPermissionlessUnstake` by 1968 ETH
6. The withdrawal request is submitted to beacon chain for 1968 ETH
7. However, the validator only has 100 ETH, so 1868 ETH of the withdrawal is **unfulfillable**

This creates an accounting discrepancy where `pendingPermissionlessUnstake` , a value stored in the `YieldManager` contract that is used to track unstaking requests, is permanently inflated by the unfulfillable amount. In turn, the `YieldManager` contract uses this variable to limit unstaking requests by users so they don’t unstake more than required by the deficit:

contracts/contracts/yield/YieldManager.sol:L570-L576

```
uint256 targetDeficit = getTargetReserveDeficit();
uint256 availableFundsToSettleTargetDeficit = address(this).balance +
    withdrawableValue(_yieldProvider) +
    _getYieldManagerStorage().pendingPermissionlessUnstake;
if (availableFundsToSettleTargetDeficit + maxUnstakeAmount > targetDeficit) {
    revert PermissionlessUnstakeRequestPlusAvailableFundsExceedsTargetDeficit();
}
```

However, in our case the stored `pendingPermissionlessUnstake` will never decrease from the unfulfillable requests, as it only gets reduced during succesful withdrawals:

contracts/contracts/yield/YieldManager.sol:L619-L629

```
function _delegatecallWithdrawFromYieldProvider(address _yieldProvider, uint256 _amount) internal {
    YieldProviderStorage storage $$ = _getYieldProviderStorage(_yieldProvider);
    _delegatecallYieldProvider(
        _yieldProvider,
        abi.encodeCall(IYieldProvider.withdrawFromYieldProvider, (_yieldProvider, _amount))
    );
    $$ .userFunds -= _amount;
    _getYieldManagerStorage().userFundsInYieldProvidersTotal -= _amount;
    // Greedily reduce pendingPermissionlessUnstake with every withdrawal made from the yield provider.
    _decrementPendingPermissionlessUnstake(_amount);
}
```

As a result, if users perform enough of such unfulfillable requests up the maximum possible amount, the `availableFundsToSettleTargetDeficit + maxUnstakeAmount > targetDeficit` check will revert for future legitimate unstake requests, causing a DoS in the permissionless unstaking feature. This can be alleviated with other withdrawals such as those by permissioned unstaking requests, but permissionless unstaking requests would be blocked until that time.

Moreover, this attack can be done by using the same proof, repeating it to hit the limit of the maximum `pendingPermissionlessUnstake` as needed.

Recommendation

As in [issue 5.3](#) , though the cause of the issue is the lack of instant synchronization between the beacon chain and the execution layer, requiring the proofs to be more recent would not solve the issue. A proof as old as 1 slot ago would still be stale, showing incorrect information. Instead, tracking needs to be implemented that ensures that any permissionless unstaking blocks any other requests that contain proofs that haven’t had the chance to update for the pending withdrawal information.

5.3 `unstakePermissionless` Forced Unstake Attack Using Valid but Stale Container Proofs That Unstake Any Validator Top-Ups or Increases in Balances Major ✓ Fixed

Resolution
Fixed in PR 1879 by: ensuring that the <code>amounts[0]</code> that is provided to <code>_unstake()</code> is the returned value from <code>_validateUnstakePermissionlessRequest()</code> instead of the user-provided <code>amounts</code> .

target deficit and the current funds meant to rebalance it.

Description

The `ValidatorContainerProofVerifier` validates beacon chain proofs against EIP-4788 beacon roots but does not enforce proof recency beyond the 8191-block (~27 hour) history window guaranteed by the beacon roots contract. This allows attackers to submit valid but stale proofs that reflect a validator’s historical low balance, while requesting unstaking of a much larger current balance. The vulnerability leads to incorrect accounting in `pendingPermissionlessUnstake`, as the system tracks only the historical difference between effective balance and activation balance (potentially very small), while the actual unstaking request withdraws the full requested amount.

The attack exploits the timing window when validators are topped up with additional ETH. An attacker can:

- Identify a validator with low balance (e.g., newly activated at 32 ETH)
- Wait for the validator to be topped up with additional funds
- Generate a proof from before the top-up (up to 27 hours old)
- Submit `unstakePermissionless` with `amounts[0]` set to the entire top-up amount
- The proof verification passes (stale but valid), computing `maxUnstakeAmountGwei = Math256.min(amounts[0], effectiveBalance - 32e9) ≈ 0`
- The system increments `pendingPermissionlessUnstake` by only the small `maxUnstakeAmount` (near zero)
- However, the actual withdrawal request proceeds with the full `amounts[0]`, withdrawing significantly more ETH than tracked
- This creates an accounting discrepancy where `availableFundsToSettleTargetDeficit + maxUnstakeAmount > targetDeficit` checks can be bypassed, allowing excessive unstaking beyond intended limits.

Repeated exploitation can:

- Bypass the permissionless unstake caps designed to prevent over-unstaking
- Create a permanent DoS on validator top-ups (any top-up gets immediately exploited and unstaked)

Examples

The `unstakePermissionless()` request validates against stale witness data but ultimately submits the user-provided `amounts` to `_unstake()`, which is not changed by `_validateUnstakePermissionlessRequest()` as opposed to `maxUnstakeAmount`:

contracts/contracts/yield/LidoStVaultYieldProvider.sol:L311-L321

```
function unstakePermissionless(
    address _yieldProvider,
    bytes calldata _withdrawalParams,
    bytes calldata _withdrawalParamsProof
) external payable onlyDelegateCall returns (uint256 maxUnstakeAmount) {
    (bytes memory pubkeys, uint64[] memory amounts, address refundRecipient) = abi.decode(
        _withdrawalParams,
        (bytes, uint64[], address)
    );
    maxUnstakeAmount = _validateUnstakePermissionlessRequest(_yieldProvider, pubkeys, amounts, _withdrawalParamsProof);
    _unstake(_yieldProvider, pubkeys, amounts, refundRecipient);
}
```

contracts/contracts/yield/LidoStVaultYieldProvider.sol:L372-L381

```
VALIDATOR_CONTAINER_PROOF_VERIFIER.verifyActiveValidatorContainer(witness, _pubkeys, withdrawalCredentials);

// https://github.com/ethereum/consensus-specs/blob/master/specs/electra/beacon-chain.md#modified-get_expected_withdrawals
uint256 maxUnstakeAmountGwei = Math256.min(
    amount,
    Math256.safeSub(witness.effectiveBalance, MIN_0X02_VALIDATOR_ACTIVATION_BALANCE_GWEI)
);
// Convert from Beacon Chain units of 'gwei' to execution layer units of 'wei'
maxUnstakeAmount = maxUnstakeAmountGwei * 1 gwei;
}
```

The returned `maxUnstakeAmount` is then validated and registered into the state on the `YieldManager` contract:

contracts/contracts/yield/YieldManager.sol:L561-L576

```
bytes memory data = _delegatecallYieldProvider(
    _yieldProvider,
    abi.encodeCall(IYieldProvider.unstakePermissionless, (_yieldProvider, _withdrawalParams, _withdrawalParamsProof))
);
maxUnstakeAmount = abi.decode(data, (uint256));
if (maxUnstakeAmount == 0) {
    revert YieldProviderReturnedZeroUnstakeAmount();
}
// Validate maxUnstakeAmount
uint256 targetDeficit = getTargetReserveDeficit();
uint256 availableFundsToSettleTargetDeficit = address(this).balance +
    withdrawableValue(_yieldProvider) +
    _getYieldManagerStorage().pendingPermissionlessUnstake;
if (availableFundsToSettleTargetDeficit + maxUnstakeAmount > targetDeficit) {
    revert PermissionlessUnstakeRequestPlusAvailableFundsExceedsTargetDeficit();
}
```

It is important to note that the full 8192 slot period is not the root cause, and even a 1 slot long period could allow for this issue. As long as the information between the beacon chain and the execution layer is not synced instantly, a user could submit a proof that is not current

Enforce that the validated `maxUnstakeAmount` that is the result of the witness proof calculation is used for the subsequent `_unstake()` request on the beacon chain to synchronize the requested withdrawal amount and the recorded amount in the `YieldManager` state.

5.4 If `unstake` Is Pending While `unstakePermissionless` Can Be Called the System May Unstake More Than the System Intends

MediumAcknowledged

Resolution
Acknowledged as acceptable. Further over-unstaking beyond the initial will not be possible as the result of the first <code>unstake</code> will decrease the deficit, reducing what can be requested for additional unstaking.

Description

The `YieldManager` contract tracks and limits how much unstaking can be requested permissionlessly via the storage variable `pendingPermissionlessUnstake` . It is incremented upon each succesfull request via `unstakePermissionless()` , and it is also checked on each subsequent call to protect against over-unstaking:

contracts/contracts/yield/YieldManager.sol:L570-L578

```
uint256 targetDeficit = getTargetReserveDeficit();
uint256 availableFundsToSettleTargetDeficit = address(this).balance +
    withdrawableValue(_yieldProvider) +
    _getYieldManagerStorage().pendingPermissionlessUnstake;
if (availableFundsToSettleTargetDeficit + maxUnstakeAmount > targetDeficit) {
    revert PermissionlessUnstakeRequestPlusAvailableFundsExceedsTargetDeficit();
}

_getYieldManagerStorage().pendingPermissionlessUnstake += maxUnstakeAmount;
```

To allow for future unstaking requests, it is decremented during `_delegatecallWithdrawFromYieldProvider()` when ETH is pulled from a yield provider to the `YieldManager` contract:

contracts/contracts/yield/YieldManager.sol:L619-L636

```
function _delegatecallWithdrawFromYieldProvider(address _yieldProvider, uint256 _amount) internal {
    YieldProviderStorage storage $$ = _getYieldProviderStorage(_yieldProvider);
    _delegatecallYieldProvider(
        _yieldProvider,
        abi.encodeCall(IYieldProvider.withdrawFromYieldProvider, (_yieldProvider, _amount))
    );
    $$userFunds -= _amount;
    _getYieldManagerStorage().userFundsInYieldProvidersTotal -= _amount;
    // Greedily reduce pendingPermissionlessUnstake with every withdrawal made from the yield provider.
    _decrementPendingPermissionlessUnstake(_amount);
}

function _decrementPendingPermissionlessUnstake(uint256 _amount) internal {
    YieldManagerStorage storage $ = _getYieldManagerStorage();
    uint256 pendingPermissionlessUnstakeAmount = $.pendingPermissionlessUnstake;
    if (pendingPermissionlessUnstakeAmount == 0) return;
    $.pendingPermissionlessUnstake = Math256.safeSub(pendingPermissionlessUnstakeAmount, _amount);
}
```

However, this function is called on every withdrawal from the yield provider, not just those withdrawals that are associated with permissionless unstaking requests. As a result, this allows additional permissionless unstakes beyond the intended limits, potentially causing excessive unstaking when reserves are already being restored through regular unstake operations.

For example, when a regular `unstake()` for X ETH is initiated first, followed by `unstakePermissionless()` calls up to the target limit, the completion of the regular `unstake()` via `withdrawFromYieldProvider()` decrements `pendingPermissionlessUnstake` by X ETH. This reduction would allow for another X ETH worth of additional permissionless unstaking capacity, bypassing the check associated with the `PermissionlessUnstakeRequestPlusAvailableFundsExceedsTargetDeficit` error that should prevent over-unstaking.

This instance of over-unstaking may happen when there is a pending `unstake()` and the system is in target deficit which would also allow to initiate `unstakePermissionless()` at the same time.

Recommendation

Consider accounting for permissioned unstaking requests when calculating the maximum amounts of permissionless unstaking requests. Consider implementing a tie-in mechanism between each unstaking request and the withdrawals that follow.

5.5 Whale-Driven Reserve Depletion Forcing Excessive LST Liability

MediumFixed

Resolution
Acknowledged. The Linea team will implement multiple operational safeguards: (1) conservatively high reserve requirements, (2) soft offchain rate limits for L1MessageService-to-staking transfers, (3) existing onchain rate limits for L1MessageService withdrawals (applies to LST withdrawals), (4) offchain rebalancing logic accounting for both reserve deficits and outstanding LST liabilities, and (5) offchain circuit breaker to stop rebalancing if the required L1MessageService-to-staking transfer is too large.

A large ETH depositor can orchestrate a reserve depletion attack by inflating system reserves with a massive deposit, waiting for automated staking to occur, then immediately withdrawing the full amount. As the balance has already been sent for staking operations, this forces the protocol to satisfy withdrawals using LST tokens via `claimMessageWithProofAndWithdrawLST()` :

contracts/contracts/LineaRollupYieldExtension.sol:L138-L153

```
function claimMessageWithProofAndWithdrawLST(
    ClaimMessageWithProofParams calldata _params,
    address _yieldProvider
) external virtual nonReentrant {
    if (_params.value < address(this).balance) {
        revert LSTWithdrawalRequiresDeficit();
    }
    if (msg.sender != _params.to) {
        revert CallerNotLSTWithdrawalRecipient();
    }
    bytes32 messageLeafHash = _validateAndConsumeMessageProof(_params);
    IS_WITHDRAW_LST_ALLOWED = true;
    IYieldManager(yieldManager()).withdrawLST(_yieldProvider, _params.value, _params.to);
    IS_WITHDRAW_LST_ALLOWED = false;
    emit MessageClaimed(messageLeafHash);
}
```

This creates substantial LST liabilities that require a sum of 1) staking queue period for the staking funds to arrive to the beacon chain and 2) partial withdrawal queue to return the funds from the beacon chain to the Message Service. During this period, LST liabilities continue growing while rewards do not get generating in the staking queue. Essentially, this attack would incur fees and liabilities on Linea staking operations but wouldn’t yield anything for the attacker.

Example attack for a system with 100,000 ETH total and a 50% reserve target.

1. Large holder deposits 200,000 ETH increasing L1MessageService balance from 50,000 to 250,000 ETH
2. Automation service stakes 100,000 ETH based on 50% target reserve, leaving 150,000 ETH in reserves
3. The large holder withdraws 200,000 ETH, depleting L1MessageService balance to zero and forcing 50,000 ETH to be fulfilled via LST withdrawals.

Recommendation

Implement defensive reserve percentage calculations based on maximum tolerable attacker balance rather than arbitrary values. Modify rebalancing logic to account for both reserve deficits and outstanding LST liabilities when determining withdrawal amounts from validators. Consider implementing rate limits on both L1MessageService-to-staking transfers and LST withdrawals to increase attack cost and cap liability growth during bank-run scenarios.

5.6 `claimMessageWithProofAndWithdrawLST` Requires `msg.sender==params._to` Which Is Inconsistent With `claimMessageWithProof` Behavior Medium ✓ Fixed

Resolution
Acknowledged. The team has confirmed this is an intended design choice to ensure recipients are aware of the LST currency conversion and potential tax implications. Requiring <code>msg.sender == params._to</code> enforces that the recipient explicitly initiates the conversion, as claiming on their behalf could result in undesired tax consequences.

Description

The LST withdrawal function `claimMessageWithProofAndWithdrawLST()` enforces that only the original to address specified in the withdrawal request can claim the funds (requiring `to == msg.sender`):

contracts/contracts/LineaRollupYieldExtension.sol:L138-L147

```
function claimMessageWithProofAndWithdrawLST(
    ClaimMessageWithProofParams calldata _params,
    address _yieldProvider
) external virtual nonReentrant {
    if (_params.value < address(this).balance) {
        revert LSTWithdrawalRequiresDeficit();
    }
    if (msg.sender != _params.to) {
        revert CallerNotLSTWithdrawalRecipient();
    }
}
```

This is in contrast to the regular ETH claiming function that allows the Message Service contract to actually perform a low level call to the recipient `to` address with an associated `value` value:

contracts/contracts/messageService/I1/L1MessageService.sol:L87-L94

```
function claimMessageWithProof(
    ClaimMessageWithProofParams calldata _params
) external nonReentrant distributeFees(_params.fee, _params.to, _params.data, _params.feeRecipient) {
    bytes32 messageLeafHash = _validateAndConsumeMessageProof(_params);

    TransientStorageHelpers.tstoreAddress(MESSAGE_SENDER_TRANSIENT_KEY, _params.from);

    (bool callSuccess, bytes memory returnData) = _params.to.call{ value: _params.value }(_params.data);
```

shortfall on the L1 Message Service due to staking operations, the users performing the message claim may not be anticipating the `to==msg.sender` requirement, as that is not the case with `claimMessageWithProof()`

This creates a risk where withdrawal requests to treasury, multisig or other smart contracts that cannot execute arbitrary function calls will become temporarily stuck, effectively locking user funds until funds are replenished on the L1 Message Service.

Recommendation

Consider relaxing the `to==msg.sender` requirement to allow claiming on behalf of the recipients.

5.7 A Timed Large-Holder Attack Can Cause the System to Over-Unstake Whenever unstake Is Called Medium ✓ Fixed

Resolution
Acknowledged. The team made a design choice not to track permissioned <code>unstake()</code> requests to enable gas efficiency for trusted actors, as validating unvalidated inputs would be unsafe for state changes. They accept that the system may unstake more than strictly required in this scenario, viewing it as acceptable since it doesn’t compromise user withdrawals. The impact is a temporary drop in yield efficiency, partially mitigated by pending partial withdrawals still earning yield.

Description

A large ETH holder can exploit the unstaking mechanism by timing large withdrawals to coincide with pending regular unstaking operations. The system calculates permissionless unstaking requirements based on current balance without accounting for incoming funds from pending regular unstaking, allowing attackers to force the protocol to over-unstake beyond intended limits. This occurs because the system doesn’t track or subtract pending incoming unstaking amounts when determining how much additional permissionless unstaking is needed to reach minimum thresholds.

The vulnerability manifests in the following attack sequence:

- System initiates regular unstaking (e.g., 5k ETH) to reach target balance via `unstake()`
- Attacker times large withdrawal (e.g., 10-30k ETH) while regular unstaking is pending
- System calculates permissionless unstaking requirement based on post-withdrawal balance, ignoring incoming regular unstaking:

contracts/contracts/yield/YieldManager.sol:L570-L576

```
uint256 targetDeficit = getTargetReserveDeficit();
uint256 availableFundsToSettleTargetDeficit = address(this).balance +
    withdrawableValue(_yieldProvider) +
    _getYieldManagerStorage().pendingPermissionlessUnstake;
if (availableFundsToSettleTargetDeficit + maxUnstakeAmount > targetDeficit) {
    revert PermissionlessUnstakeRequestPlusAvailableFundsExceedsTargetDeficit();
}
```

- When regular unstaking completes, `pendingPermissionlessUnstake` is decremented via `safeSub` but incoming funds push balance above minimum
- Result: System has over-unstaked by the amount of the original regular unstaking

Example scenario: With 100k ETH total funds (40% min reserve, 50% target reserve) and 50k ETH on the L1 Message Service, after a 10k ETH withdrawal and 5k ETH regular unstaking initiated, a subsequent 30k ETH withdrawal may trigger a 14k ETH permissionless unstaking. When the original 5k ETH regular unstaking completes, total unstaking becomes 19k ETH instead of the required 14k ETH, exceeding intended limits by 5k ETH.

Recommendation

Consider implementing tracking of pending regular unstaking amounts to tie all withdrawal operations with associated unstaking requests. Consequently, account for all unstaking requests amounts to prevent over-unstaking.

5.8 Over-Unstaking Due to Greedy Decrementing of pendingPermissionlessUnstake Medium ✓ Fixed

Resolution
Acknowledged. The team has made a design trade-off with the greedy decrement approach: the system cannot distinguish whether StakingVault funds originate from unstaking completions, staking yield, or direct deposits. This approach prioritizes maintaining user withdrawal liquidity over perfect yield efficiency.

Description

The Yield Manager decrements `pendingPermissionlessUnstake` whenever funds are withdrawn from the Staking Vault to the L1 reserve, incorrectly assuming these withdrawals represent completed unstaking operations. However, withdrawal operations extract liquid funds already available in the Staking Vault, not funds arriving from beacon chain unstaking.

This creates an accounting mismatch: the system tracks fewer pending unstake funds than actually exist, allowing additional permissionless unstaking that exceeds intended thresholds. The actual unstaked funds remain pending on the beacon chain while the system permits new unstake operations.

Metric	Value
Target deficit	400 ETH
Yield Manager balance	100 ETH
Staking Vault liquid balance	100 ETH
pendingPermissionlessUnstake	200 ETH
availableFundsToSettleTargetDeficit	400 ETH

Action: `withdrawFromYieldProvider(100)` extracts 100 ETH liquid funds from Staking Vault to L1 reserve

Resulting State:

Metric	Value	Change
Target deficit	300 ETH	-100 (funds arrived at L1)
Yield Manager balance	100 ETH	-
Staking Vault liquid balance	0 ETH	-100
pendingPermissionlessUnstake	100 ETH	-100
availableFundsToSettleTargetDeficit	200 ETH	-200

Issue: The system now permits an additional 100 ETH of permissionless unstaking, but the original 200 ETH from permissionless unstaking is still pending on the beacon chain. The withdrawn 100 ETH was pre-existing liquid balance, not unstaking proceeds.

Impact: Permissionless unstaking exceeds intended thresholds by the amount of liquid funds withdrawn, resulting in reduced yield efficiency.

The greedy decrement approach is a design trade-off: the system cannot distinguish whether Staking Vault funds originate from unstaking completions, staking yield, or direct deposits. This approach prioritizes the core objective of maintaining user withdrawal liquidity over perfect yield efficiency.

5.9 Inconsistent `_pauseStakingIfNotAlready` Conditions in Withdrawal Functions Medium ✓ Fixed

Resolution
Acknowledged. The team has opted for a conservative approach, prioritizing withdrawal availability over yield efficiency. They prefer to pause staking unnecessarily rather than risk allowing staking when funds may be needed for withdrawals. Additional pause logic added to <code>fundYieldProvider()</code> in PR1945 .

Description

The system has inconsistent logic for determining when to pause staking across different withdrawal functions, leading to scenarios where staking is paused unnecessarily despite sufficient liquid funds being available.

Expected Behavior: `replenishWithdrawalReserve` correctly considers both Yield Manager balance and Yield Provider balance when determining if liquid funds can cover the target deficit, only pausing staking when these combined funds are insufficient.

Inconsistent Implementations:

1. `withdrawFromYieldProvider` **Issue:** This function ignores the Yield Manager balance entirely. It withdraws only the caller-specified amount from the Yield Provider and pauses staking if the amount sent to the L1_Message_service reserve is less than the target deficit.

Problematic scenarios:

- **Scenario A:** Substantial funds exist in the Yield Manager, but the Yield Provider balance alone is less than the target deficit. Staking is paused despite total liquid funds being sufficient.
- **Scenario B:** Caller passes an amount smaller than the Yield Provider’s available balance, even though the full Yield Provider balance could cover the deficit. Staking is paused unnecessarily.

2. `_addToWithdrawalReserve` **Issue:** This function pauses staking if the input amount is less than the target deficit. The behavior differs based on the caller:

- When called via `safeAddToWithdrawalReserve` : Safe because it considers full Yield Manager and Yield Provider balances
- When called via `addToWithdrawalReserve` : Vulnerable because it only considers the input amount, enabling a frontrunning attack where an attacker withdraws from L1 reserve to increase the deficit, causing the function to incorrectly pause staking despite sufficient liquid funds remaining

Impact: Staking may be paused prematurely in situations where combined liquid funds (Yield Manager + Yield Provider) are sufficient to cover the target deficit, reducing yield generation without justification.

Recommendation: All withdrawal functions should consistently evaluate total liquid funds (Yield Manager balance + Yield Provider balance) before determining whether to pause staking.

5.10 Lido External Shares Limit May Block Emergency LST Withdrawals Medium ✓ Fixed

Resolution

constrained by the vendor. Users will still be able to access their funds but will have to wait for validator withdrawals in such cases.

Description

Lido’s 20% external shares limit may prevent emergency LST withdrawals during liquidity crises, forcing users into slower beacon chain withdrawal queues when immediate liquidity is most needed.

Lido caps external shares at 20% of total stETH supply via `_getMaxMintableExternalShares()` . The Linea Yield Management system relies on LST minting as emergency liquidity when `L1MessageService` reserves are insufficient, executed through `claimMessageWithProofAndWithdrawLST()` → `withdrawLST()` → `IDashboard.mintStETH()` .

Risk Scenario

As other L2s adopt similar yield strategies, external shares utilization could approach the 20% limit through:

- Capital migration from traditional stETH to L2 yield strategies
- Multiple L2s competing for the same external shares allocation
- Composition shifts even without net new demand

When the limit is reached, Lido rejects new stETH minting requests, denying users emergency liquidity access.

Impact

- Emergency liquidity denial during withdrawal crises
- Unpredictable LST availability undermining system reliability
- Forced beacon chain queues (days/weeks delay) during stressed conditions

This represents a systemic constraint outside the protocol’s direct control that could impact emergency liquidity provision.

5.11 `_hashMessageWithEmptyCallldata` Does an Unnecessary 0x Byte Insert Minor ✓ Fixed

Resolution
Fixed in commit cdfb9c3d0781dc5e3f02de8026c6063b07330838 .

Description

The `_hashMessageWithEmptyCallldata` function performs an unnecessary memory store operation at position `0xe0` before hashing. Since the `keccak256` operation only hashes up to position `0xe0` (exclusive), storing data at that position serves no purpose and wastes gas.

`contracts/contracts/messageService/lib/MessageHashing.sol:L58-L77`

```
function _hashMessageWithEmptyCallldata(
    address _from,
    address _to,
    uint256 _fee,
    uint256 _valueSent,
    uint256 _messageNumber
) internal pure returns (bytes32 messageHash) {
    assembly {
        let mPtr := mload(0x40)
        mstore(mPtr, _from)
        mstore(add(mPtr, 0x20), _to)
        mstore(add(mPtr, 0x40), _fee)
        mstore(add(mPtr, 0x60), _valueSent)
        mstore(add(mPtr, 0x80), _messageNumber)
        mstore(add(mPtr, 0xa0), 0xc0)
        mstore(add(mPtr, 0xc0), 0x00)
        mstore(add(mPtr, 0xe0), 0x00)
        messageHash := keccak256(mPtr, 0xe0)
    }
}
```

Recommendation

Remove the unnecessary `mstore(add(mPtr, 0xe0), 0x00)` operation to optimize gas usage.

5.12 Unnecessarily Tight Boundary on a Check When Withdrawing With LST Tokens Minor ✓ Fixed

Resolution
Fixed in commit 9722a2a048c9e0c0f0dd8bf959a37516e7590b8d .

Description

When withdrawing from the bridge with LST tokens via `claimMessageWithProofAndWithdrawLST()` , we require the requested amount to not be strictly less than the balance of the message service to enable this route:

```
ClaimMessageWithProofParams calldata _params,
address _yieldProvider
) external virtual nonReentrant {
    if (_params.value < address(this).balance) {
        revert LSTWithdrawalRequiresDeficit();
    }
}
```

The idea is that if there is enough to satisfy on the bridge, LST withdrawal shouldn’t happen, as per the comment “...withdraws LST from the specified yield provider when the L1MessageService balance is insufficient to fulfill delivery” However, if there is an equal amount, that is also sufficient, and in this case the LST withdrawal is still permissible. In other words, the check should be `<=` instead of `<`.

Recommendation

Adjust the inequality check to `<=`.

5.13 Unnecessary O-Address Checks Minor ✓ Fixed

Resolution
Fixed in commit 1b682ff4ad2cf4ccf7ff61dcc521241dcee1a13d .

Description

`YieldProviderBase` does O-address checks for `_l1MessageService` and `_yieldManager` in its constructor:

contracts/contracts/yield/YieldProviderBase.sol:L21-L23

```
constructor(address _l1MessageService, address _yieldManager) {
    ErrorUtils.revertIfZeroAddress(_l1MessageService);
    ErrorUtils.revertIfZeroAddress(_yieldManager);
}
```

However, the child contract `LidoStVaultYieldProvider` does the same checks as well, which is unnecessary:

contracts/contracts/yield/LidoStVaultYieldProvider.sol:L84-L93

```
constructor(
    address _l1MessageService,
    address _yieldManager,
    address _vaultHub,
    address _vaultFactory,
    address _steth,
    address _validatorContainerProofVerifier
) YieldProviderBase(_l1MessageService, _yieldManager) {
    ErrorUtils.revertIfZeroAddress(_l1MessageService);
    ErrorUtils.revertIfZeroAddress(_yieldManager);
}
```

Recommendation

Remove unnecessary O-address checks in the `LidoStVaultYieldProvider` contract.

5.14 Inconsistent O-Address Checking for `_yieldManager` Minor ✓ Fixed

Resolution
Fixed in commit a7129325567e6435a81b8bbdce7507a803a01be3 .

Description

`LineaRollupYieldExtension` does not check if `_yieldManager` is 0 in its `__LineaRollupYieldExtension_init()` function, as that is done in the `LineaRollup` `initialize()`.

contracts/contracts/LineaRollupYieldExtension.sol:L51-L54

```
function __LineaRollupYieldExtension_init(address _yieldManager) internal onlyInitializing {
    emit YieldManagerChanged(_storage()._yieldManager, _yieldManager);
    _storage()._yieldManager = _yieldManager;
}
```

contracts/contracts/LineaRollup.sol:L23-L29

```
function initialize(InitializationData calldata _initializationData) external initializer {
    __LineaRollup_init(_initializationData);
    if (_initializationData.initialYieldManager == address(0)) {
        revert ZeroAddressNotAllowed();
    }
    __LineaRollupYieldExtension_init(_initializationData.initialYieldManager);
}
```

contracts/contracts/LineaRollupYieldExtension.sol:L87-L92

```
function setYieldManager(address _newYieldManager) public onlyRole(SET_YIELD_MANAGER_ROLE) {
    require(_newYieldManager != address(0), ZeroAddressNotAllowed());
    LineaRollupYieldExtensionStorage storage $ = _storage();
    emit YieldManagerChanged($._yieldManager, _newYieldManager);
    $_yieldManager = _newYieldManager;
}
```

It would be more consistent to perform all 0-address checks for the `_yieldManager` variable within the `LineaRollupYieldExtension` contract.

Recommendation

Move the 0-address check for the `_yieldManager` variable from `LineaRollup.initialize()` to `LineaRollupYieldExtension._LineaRollupYieldExtension_init()`

5.15 Claiming ETH via the LST Route Does Not Burn LINEA ✓ Fixed

Resolution
Acknowledged. The team has noted that L1LineaTokenBurner changes are out-of-scope for this release.

Description

The Linea rollup has recently introduced a feature into its message claiming flow where a claimed message could burn `LINEA` tokens from the `L1LineaTokenBurner` contract:

contracts/src/operational/L1LineaTokenBurner.sol:L31-L36

```
/**
 * @notice Claims a message with proof and burns the LINEA tokens held by this contract.
 * @dev This is expected to be permissionless, allowing anyone to trigger the burn.
 * @param _params The parameters required to claim the message with proof.
 */
function claimMessageWithProof(IL1MessageService.ClaimMessageWithProofParams calldata _params) external {
```

In the feature set of the scope of the current audit, there is also a new functionality introduced into the message claiming flow - `claimMessageWithProofAndWithdrawLST`. In the event of a liquidity shortfall on the L1 Message Service contract due to staking operations, users may utilize this function to withdraw `stETH` instead of `ETH`, effectively offering an alternative to a normal `claimMessageWithProof` function.

As a result, for full feature parity it may be worth it to consider introducing a call (with associated necessary changes) to this function in the `L1LineaTokenBurner` contract as well, so that the `LINEA` tokens may be burnt through this flow as well.

Recommendation

Consider the `claimMessageWithProofAndWithdrawLST` function to enable `LINEA` burning via `L1LineaTokenBurner`.

5.16 Some `@dev` Comments Explicitly Call Out the Required Role to Call the Function, Some Don't

✓ Fixed

Resolution
Fixed in commit da16632b6ec1942b5bf8fb471390e07ee24e5d1d .

Description

Protected functions in the YieldManager contract have inconsistent natspec documentation regarding required roles. Some functions properly document the required role in their `@dev` comments while others don't.

Examples

- `reportYield` with `@dev` :

contracts/contracts/yield/YieldManager.sol:L470-L472

```
/**
 * @notice Report newly accrued yield for the YieldProvider since the last report.
 * @dev YIELD_REPORTER_ROLE is required to execute.
```

- `initiateOssification` without:

contracts/contracts/yield/YieldManager.sol:L870-L876

```
* @notice Initiate the ossification sequence for a provider.
* @dev Will pause beacon chain staking and LST withdrawals.
* @dev WARNING: This operation irreversibly pauses beacon chain deposits.
* @param _yieldProvider The yield provider address.
*/
function initiateOssification(
```

Recommendation

Adjust @dev comments as needed.

6 Self Reported by the Linea Team

6.1 Stale withdrawableValue Calculation Blocks All Withdrawals When LST Liabilities Exist Major

✓ Fixed

Resolution
Fixed in PR 1849 . The team introduced a <code>beforeWithdrawFromYieldProvider</code> hook that enables the <code>LidoStVaultYieldProvider</code> to settle LST liabilities before withdrawals, preventing stale <code>withdrawableValue</code> calculations. Settlement is appropriately skipped during permissionless reserve deficit withdrawals.

Description

This is a self-reported issue found by the Linea team. When LST liabilities exist in the system, all withdrawal operations from the Yield Manager become blocked due to a stale withdrawable value calculation.

Intended Flow:

The Yield Manager’s withdrawal functions follow this pattern:

- 1. Query `dashboard.withdrawableValue()` to determine how much can be withdrawn
- 2. Pass that value to `dashboard.withdraw()` to execute the withdrawal

Actual Behavior:

The issue occurs when LST liabilities are present in the system. The withdrawal flow executes as follows:

- 1. Yield Manager calls `dashboard.withdrawableValue()` and stores the result (e.g., 1000 ETH)
- 2. Yield Manager calls `dashboard.withdraw(1000)`
- 3. This triggers `LidoStVaultYieldProvider::withdrawFromYieldProvider`
- 4. **Before the actual withdrawal**, `_payMaximumPossibleLSTLiability()` is called
- 5. This payment reduces the StakingVault’s available funds
- 6. The withdrawable value drops (e.g., from 1000 ETH to 800 ETH)
- 7. The system attempts to withdraw the original 1000 ETH
- 8. The Dashboard reverts with `ExceedsWithdrawable` because only 800 ETH is now available

Exploit Path:

- 1. Attacker calls `withdrawLST` with any non-zero amount to create LST liabilities
- 2. All subsequent withdrawal operations fail, blocking legitimate withdrawals from the L1 reserve

Root Cause:

The withdrawable value is calculated before the LST liability payment occurs. Since the liability payment reduces available funds, the calculated withdrawal amount becomes stale and exceeds what is actually withdrawable, causing the transaction to revert.

Impact:

Denial of service for all Yield Manager withdrawal operations when LST liabilities exist. This prevents the system from replenishing the L1 reserve, potentially blocking user withdrawals.

6.2 Missing Zero-Balance Check in progressPendingOssification Blocks Ossification Completion

Minor

✓ Fixed

Resolution
The Linea team has implemented a fix in commit bc1523b931d2f30330900577f5c4b0d78e30b946 , which skips the unstaging call when there is no balance to unstage.

Description

This is a self-reported issue found by the Linea team during testing. When `progressPendingOssification` is called, there was no check to verify that `stagedBalance` was non-zero before calling the `unstage` function. If `stagedBalance` is zero, the `unstage` call would revert, blocking full ossification from completing.

Appendix 1 - Files in Scope

File	SHA-1 hash
contracts/contracts/messageService/lib/MessageHashing.sol	8fd4ab0a3223d24838ca404271386e9c8dd64b1b
contracts/contracts/LineaRollupYieldExtension.sol	55e5680a7a85218736eb698f8158e5cd71ff26bc
contracts/contracts/yield/YieldManager.sol	3896d3baa6723a42177303f2c6e45489856e4ea6
contracts/contracts/yield/YieldManagerStorageLayout.sol	a66395e1aa83287efe05c3bc6c486c9746fb284b
contracts/contracts/yield/YieldProviderBase.sol	29724dd448ae7dfbb406d85162b8bce66b791198
contracts/contracts/yield/LidoStVaultYieldProvider.sol	e1f2d4e32ffd499a44446d7e98f2c3e28704e963
contracts/contracts/yield/LidoStVaultYieldProviderFactory.sol	f5404600d213758f9f312911bcc166729475b209
contracts/contracts/lib/YieldManagerPauseManager.sol	5d499aa2faaf93e7a2b8eee7d7e58725744490fe
contracts/contracts/lib/ErrorUtils.sol	670d40161ea86e81501b70bed6449626277ec9fa
contracts/contracts/lib/Math256.sol	0eee5665e2ff056b4f0eb977d849f967a69f9618
contracts/contracts/lib/LineaRollupPauseManager.sol	7b2cb7ba70b9478490b4cac0d4908ff09e5cf071

Appendix 2 - Disclosure

Consensys Diligence (“CD”) typically receives compensation from one or more clients (the “Clients”) for performing the analysis contained in these reports (the “Reports”). The Reports may be distributed through other means, including via Consensys publications and other distributions.

The Reports are not an endorsement or indictment of any particular project or team, and the Reports do not guarantee the security of any particular project. This Report does not consider, and should not be interpreted as considering or having any bearing on, the potential economics of a token, token sale or any other product, service or other asset. Cryptographic tokens are emergent technologies and carry with them high levels of technical risk and uncertainty. No Report provides any warranty or representation to any third party in any respect, including regarding the bug-free nature of code, the business model or proprietors of any such business model, and the legal compliance of any such business. No third party should rely on the Reports in any way, including for the purpose of making any decisions to buy or sell any token, product, service or other asset. Specifically, for the avoidance of doubt, this Report does not constitute investment advice, is not intended to be relied upon as investment advice, is not an endorsement of this project or team, and it is not a guarantee as to the absolute security of the project. CD owes no duty to any third party by virtue of publishing these Reports.

A.2.1 Purpose of Reports

The Reports and the analysis described therein are created solely for Clients and published with their consent. The scope of our review is limited to a review of code and only the code we note as being within the scope of our review within this report. Any Solidity code itself presents unique and unquantifiable risks as the Solidity language itself remains under development and is subject to unknown risks and flaws. The review does not extend to the compiler layer, or any other areas beyond specified code that could present security risks. Cryptographic tokens are emergent technologies and carry with them high levels of technical risk and uncertainty. In some instances, we may perform penetration testing or infrastructure assessments depending on the scope of the particular engagement.

CD makes the Reports available to parties other than the Clients (i.e., “third parties”) on its website. CD hopes that by making these analyses publicly available, it can help the blockchain ecosystem develop technical best practices in this rapidly evolving area of innovation.

A.2.2 Links to Other Web Sites from This Web Site

You may, through hypertext or other computer links, gain access to web sites operated by persons other than Consensys and CD. Such hyperlinks are provided for your reference and convenience only, and are the exclusive responsibility of such web sites’ owners. You agree that Consensys and CD are not responsible for the content or operation of such Web sites, and that Consensys and CD shall have no liability to you or any other person or entity for the use of third party Web sites. Except as described below, a hyperlink from this web Site to another web site does not imply or mean that Consensys and CD endorses the content on that Web site or the operator or operations of that site. You are solely responsible for determining the extent to which you may use any content at any other web sites to which you link from the Reports. Consensys and CD assumes no responsibility for the use of third-party software on the Web Site and shall have no liability whatsoever to any person or entity for the accuracy or completeness of any outcome generated by such software.

A.2.3 Timeliness of Content

The content contained in the Reports is current as of the date appearing on the Report and is subject to change without notice unless indicated otherwise, by Consensys and CD.



